

Invariantes Lean 4 de la cronología

Modelo Lean 4 de la cronología de la historia (eventos, fechas, lugares, personajes y fechas de nacimiento) generado desde la story bible; cuatro invariantes con prueba de corrección, y el caso real del eval temporal.

Generado por presentacion/build/build_annexes.py el 2026-09-24 desde el commit 78548f3. Idioma: español; los ficheros fuente en inglés (código, specs y README técnicos) se reproducen tal cual.

Modelo, invariantes y cómo lo usa el harness

Fuente: `formal/lean/README.md`

Lean 4: formal check of the story chronology

Spec 012 (block B8 of programme 004), requirements L01-L04. Lean 4 checks the **story's** chronology, not the harness's code: a Lean file generated from the novel's chronology table is compiled with `lake build`, and the build only succeeds if every invariant holds.

Core Lean only. No Mathlib, no dependencies: the pinned toolchain is all it needs.

What is modelled

`Chronology/Basic.lean` defines:

Type	Fields
Date	<code>y m d : Nat</code> (proleptic Gregorian), <code>Date.dayNumber</code> (days since 0000-03-01)
Character	<code>id : Nat</code> , <code>birth : Option Date</code>
Place	<code>id : Nat</code>
Event	<code>id</code> , <code>seq</code> (position in the telling), <code>date</code> , <code>day</code> (= <code>date.dayNumber</code> , precomputed), <code>place</code> , <code>participants : List Nat</code> , <code>kind : normal \ death \ departure</code> , <code>declaredAges : List (Nat × Nat)</code>
Story	<code>characters</code> , <code>places</code> , <code>events</code>

Ids are `Nat` indices. The exporter maps the database's string ids to indices and keeps the original ids and names in tables (`characterTable`, `placeTable`, `eventTable`) that the checks never read, so no text from the novel is ever Lean code.

The four invariant

Each is a `Bool` check that `decide` evaluates and a `Prop` saying what it means, linked by a soundness theorem, so a passing build is a proof of the `Prop`.

Invariant	Prop	Holds when
<code>temporalOrder</code>	<code>TemporalOrder</code>	every event told later (greater <code>seq</code>) happens on the same day or later

Invariant	Prop	Holds when
agesCoherent	AgesCoherent	every declared age equals the full years between the character's birth and the event date (no birth date: nothing to check)
noBilocation	NoBilocation	no character is in two different places on the same day
noAfterExit	NoAfterExit	after a death or departure event, its first participant takes part in no event told later

datesConsistent (every day equals `date.dayNumber`) is checked inside `temporalOrder` and `noBilocation`, which compare `day`.

The generated file

`backend/app/formal/lean_export.py` writes `Chronology/Generated/Story.lean` (git-ignored, regenerated on every run) with `def story : Story`, and:

```
theorem story_temporalOrder : temporalOrderB story = true := by decide +kernel
theorem story_agesCoherent : agesCoherentB story = true := by decide +kernel
theorem story_noBilocation : noBilocationB story = true := by decide +kernel
theorem story_noAfterExit : noAfterExitB story = true := by decide +kernel
theorem story_ok : validStory story = true := by simp [validStory, story_temporalOrder, ...]
theorem story_valid : ValidStory story := validStory_sound story story_ok
```

A false invariant is a build error on the theorem that names it:

```
error: Chronology/Generated/Story.lean:49:62: Tactic `decide` proved that the proposition
  agesCoherentB story = true
is false
```

`decide +kernel` rather than plain `decide`: plain `decide` first reduces the check in the elaborator, which hits `maxRecDepth` beyond about a hundred events. `native_decide` is not used, since it trusts the compiler rather than the kernel.

Cost. Build time for the generated module, on the development container: 5 events 0.6 s, 100 events 3 s, 300 events 17 s. `temporalOrder` and `agesCoherent` are linear; `noAfterExit` is $\text{exits} \times \text{events}$; `noBilocation` is events^2 and dominates.

Install and run

```
## elan (the Lean version manager) and the toolchain pinned in lean-toolchain
curl -sSfL https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh \
  | sh -s -- -y --default-toolchain none
## or, without running a shell script: the release binary
curl -sSfL -o elan.tar.gz \
  https://github.com/leanprover/elan/releases/latest/download/elan-x86_64-unknown-linux-gnu.tar.gz
tar xzf elan.tar.gz && ./elan-init -y --default-toolchain none

export PATH="$HOME/.elan/bin:$PATH"
cd formal/lean
lake build # the library; fetches the pinned toolchain once
```

Check a chronology by hand:

```
cd backend
uv run python -c "import json; from app.formal import verify_chronology as v; \
```

```
r = v(json.load(open('../formal/lean/examples/incoherent.json')), 'demo'); \
print(r.passed, r.failed_invariants)"
## False ['agesCoherent', 'noBilocation']
```

examples/ok.json passes; examples/incoherent.json declares an age one day before the birthday and puts a character in two cities on one day.

How the harness calls it

- **Input.** The chronology JSON of contract K1 (`chronology_json`, from the story bible): characters with `birth_date`, places, events with `seq`, `story_date`, `place_id`, `participants`, `kind` and `declared_ages`.
- **API.** `verify_chronology(chronology, novel_id, lean_dir=None, timeout=300) -> LeanResult(passed, failed_invariants, output, lean_file)`. It never raises: a malformed chronology, a timeout or a missing toolchain is `passed=False` with the reason in `output` (a missing lake says "Lean toolchain missing").
- **Gate.** It runs at `pre_publish` (spec 004, decision D4), registered as a validator of the protocol K3. `passed=False` blocks the version: it is not published, and the failed invariants with Lean's message go back to the editor as feedback, naming the events involved.
- **Observability.** The result is the Langfuse score `lean_chronology` (1 pass, 0 fail) on the novel's trace, with `failed_invariants` as its comment (contract K2).

The registration as a validator and the score are wired by the pipeline and validator blocks (B3, B4, B6); this block provides `verify_chronology`.

L04 — a case Lean caught that no other validator did

None yet, and this is why: at the time of writing (2026-09-24) no real novel has been exported, because the chronology table the exporter reads (K1, block B1) and the pre-publish hook (K4, block B3) are being built in parallel. The samples show the class of error Lean is there for — an age off by one around a birthday and a bilocation across chapters — which the per-scene audit cannot see, since each scene is correct on its own. The first real run through the gate (block B11's evals) records its result here: the case caught, or the statement that Lean agreed with every other validator on it.

Caso real (L04)

Fuente: docs/process/lean-caso-real.md · evals/results

Pendiente. Todavía no hay un caso real que Lean haya cazado en pre_publish y ningún otro validador (requisito L04). Cuando exista, se documenta en docs/process/lean-caso-real.md y este anexo lo incluye al reconstruirse.

Lo más cercano observado. En el eval b4-temporal (trampas temporales plantadas: edades imposibles, una mascota que muere y reaparece, una persona que emigra y vuelve, bilocación), el diagnóstico de cronología del plan (chronology_problems, los mismos cuatro invariantes que el modelo Lean, ejecutado antes de escribir) detectó las apariciones tras la muerte (noAfterExit); tras un replan el pipeline se detuvo con plan_limit sin escribir ningún capítulo. Extracto del log:

```
[19:58:00] plan problem: chronology noAfterExit: El evento e3 del capítulo 1 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e3 del capítulo 1 hace aparecer a Trueno, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e4 del capítulo 2 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e5 del capítulo 3 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e6 del capítulo 1 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e7 del capítulo 3 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e8 del capítulo 1 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
[19:58:00] plan problem: chronology noAfterExit: El evento e9 del capítulo 3 hace aparecer a Andrés Olmedo, que ya había salido de la historia (death) en e2 (capítulo 2).
```

— b4-temporal

Trampa	Invariante Lean que debería saltar	Otra defensa
Edad declarada en un recuerdo incompatible con la fecha de nacimiento	agesCoherent	judge_novel
La mascota muere y aparece en una boda años después	noAfterExit (evento death)	judge_novel
Una persona emigra "y no ha vuelto" y aparece en una barbacoa posterior	noAfterExit (evento departure)	judge_novel
Mismo día en dos ciudades	noBilocation	judge_novel

Si el planner copia las trampas a la cronología, chronology_problems lo detecta ya en el plan (replan) o lean_chronology en pre_publish (bloqueo y ronda de reparación). Si el planner las **repara en silencio**, la historia queda coherente: se registrará como "no detectado por Lean porque no llegó a la cronología", no como éxito de Lean. **Resultado:** pendiente. Aquí se anota el caso real que Lean atrape y ningún otro validador (L04), o por qué no hubo ninguno.

Modelo e invariantes en Lean 4

Fuente: `formal/lean/Chronology/Basic.lean`

```
/-!  
# Chronology of a story – model and invariants  
  
Spec 012 (block B8 of programme 004), requirements L01-L04.  
  
A story's chronology is a list of events. Each event has a discourse position (`seq`, the  
order in which the novel tells it), a story date, a place, the characters present, a kind  
(`normal`, `death`, `departure`) and the ages the text declares for some participants.  
  
Characters and places are `Nat` indices: the exporter  
(`backend/app/formal/lean_export.py`) maps the database's string ids to indices and keeps  
the original ids and names in a string table the checks never read. Keeping the checked  
model on `Nat` means every check reduces in the kernel with GMP-accelerated arithmetic,  
so `by decide` is the proof.  
  
Every invariant exists twice:  
  
* a `Bool` check (`temporalOrderB`, ...) that `decide` evaluates, and  
* a `Prop` statement (`TemporalOrder`, ...) that says what the check means,  
  
linked by a soundness theorem (`temporalOrder_sound`, ...), so a passing check is a proof  
of the `Prop`, not merely a `true`.  
-/  
  
namespace Chronology  
  
/-- A proleptic Gregorian calendar date. The exporter only emits valid dates, year  $\geq 1$ . -/  
structure Date where  
  y : Nat  
  m : Nat  
  d : Nat  
  deriving Repr, DecidableEq  
  
/-- Day number of a date (days since 0000-03-01), Howard Hinnant's `days_from_civil`  
restricted to years  $\geq 1$ , where every subtraction below is non-negative. -/  
def Date.dayNumber (t : Date) : Nat :=  
  let y := if t.m  $\leq$  2 then t.y - 1 else t.y  
  let era := y / 400  
  let yoe := y - era * 400  
  let mp := (t.m + 9) % 12  
  let doy := (153 * mp + 2) / 5 + t.d - 1  
  let doe := yoe * 365 + yoe / 4 - yoe / 100 + doy  
  era * 146097 + doe  
  
/-- Full years elapsed from `birth` to `t`: the age a person born on `birth` has on `t`. -/  
def fullYears (birth t : Date) : Nat :=  
  if t.m < birth.m  $\vee$  (t.m = birth.m  $\wedge$  t.d < birth.d) then  
    t.y - birth.y - 1  
  else  
    t.y - birth.y  
  
inductive Kind where  
  | normal  
  | death  
  | departure  
  deriving Repr, DecidableEq  
  
/-- `true` for the kinds after which the first participant leaves the story. -/  
def Kind.isExit : Kind  $\rightarrow$  Bool  
  | .normal => false  
  | .death => true
```

```

| .departure => true

structure Character where
  id : Nat
  birth : Option Date
  deriving Repr

structure Place where
  id : Nat
  deriving Repr

structure Event where
  id : Nat
  seq : Nat
  date : Date
  /-- `date.dayNumber`, precomputed by the exporter: the kernel shares no work, so the
  pairwise checks compare this field instead of recomputing the calendar  $n^2$  times.
  `datesConsistent` checks it, linearly, inside every check that relies on it. -/
  day : Nat
  place : Nat
  participants : List Nat
  kind : Kind
  /-- `(character, declared age)` pairs. -/
  declaredAges : List (Nat × Nat)
  deriving Repr

structure Story where
  characters : List Character
  places : List Place
  events : List Event
  deriving Repr

/-- Birth date of a character, if the character is known and has one. -/
def Story.birthOf (s : Story) (c : Nat) : Option Date :=
  (s.characters.find? (·.id == c)).bind (·.birth)

/-- Every event's precomputed `day` is its date's day number. -/
def datesConsistent (s : Story) : Bool :=
  s.events.all fun e => e.day == e.date.dayNumber

theorem day_eq_of_consistent {s : Story} (h : datesConsistent s = true) {e : Event}
  (he : e ∈ s.events) : e.day = e.date.dayNumber := by
  simp only [datesConsistent, List.all_eq_true] at h
  simpa using h e he

/-! ## Invariant 1 – temporal order

Events told later (greater `seq`) never happen earlier in story time.

The check is linear: the exporter lists events by `seq`, and `seqChain` checks each
adjacent pair has a strictly greater `seq` and a day no earlier. `seqChain_pairwise` lifts
that to every pair, so the `Prop` quantifies over all pairs, whatever the list order the
check was given. (A pairwise check costs the kernel about 200  $\mu$ s a pair.) -/

/-- The ordering an adjacent pair must satisfy. -/
def Before (a b : Event) : Prop := a.seq < b.seq ∧ a.day ≤ b.day

def seqChain : List Event → Bool
| a :: b :: rest => decide (a.seq < b.seq) && decide (a.day ≤ b.day) && seqChain (b :: rest)
| _ => true

theorem seqChain_pairwise : ∀ l : List Event, seqChain l = true → l.Pairwise Before
| [], _ => List.Pairwise.nil
| [_, _] => List.pairwise_singleton _ _
| a :: b :: rest, h => by
  simp only [seqChain, Bool.and_eq_true, decide_eq_true_eq] at h
  obtain ({hs, hd}, hr) := h

```

```

have ih := seqChain_pairwise (b :: rest) hr
refine List.Pairwise.cons ?_ ih
intro x hx
cases List.mem_cons.mp hx with
| inl hxb => subst hxb; exact {hs, hd}
| inr hxr =>
  have hbx := (List.pairwise_cons.mp ih).1 x hxr
  exact {Nat.lt_trans hs hbx.1, Nat.le_trans hd hbx.2}

theorem day_le_of_pairwise : ∀ {l : List Event}, l.Pairwise Before →
  ∀ {a b : Event}, a ∈ l → b ∈ l → a.seq < b.seq → a.day ≤ b.day
| [], _, _, _, ha, _, _ => absurd ha (List.not_mem_nil)
| x :: xs, hp, a, b, ha, hb, hlt => by
  have hx := (List.pairwise_cons.mp hp).1
  have hxs := (List.pairwise_cons.mp hp).2
  cases List.mem_cons.mp ha with
  | inl hax =>
    cases List.mem_cons.mp hb with
    | inl hbx => subst hax; subst hbx; exact absurd hlt (Nat.lt_irrefl _)
    | inr hbxs => subst hax; exact (hx b hbxs).2
  | inr haxs =>
    cases List.mem_cons.mp hb with
    | inl hbx => subst hbx; exact absurd (Nat.lt_trans hlt (hx a haxs).1) (Nat.lt_irrefl _)
    | inr hbxs => exact day_le_of_pairwise hxs haxs hbxs hlt

def temporalOrderB (s : Story) : Bool :=
  datesConsistent s && seqChain s.events

def TemporalOrder (s : Story) : Prop :=
  ∀ a ∈ s.events, ∀ b ∈ s.events, a.seq < b.seq → a.date.dayNumber ≤ b.date.dayNumber

theorem temporalOrder_sound (s : Story) (h : temporalOrderB s = true) : TemporalOrder s := by
  intro a ha b hb hlt
  simp only [temporalOrderB, Bool.and_eq_true] at h
  rw [← day_eq_of_consistent h.1 ha, ← day_eq_of_consistent h.1 hb]
  exact day_le_of_pairwise (seqChain_pairwise _ h.2) ha hb hlt

/-! ## Invariant 2 – declared ages agree with birth dates

A declared age equals the full years between the character's birth and the event date.
A character with no recorded birth date constrains nothing. -/

def ageOk (s : Story) (e : Event) (ca : Nat × Nat) : Bool :=
  match s.birthOf ca.1 with
  | none => true
  | some b => ca.2 == fullYears b e.date

def agesCoherentB (s : Story) : Bool :=
  s.events.all fun e => e.declaredAges.all fun ca => ageOk s e ca

def AgesCoherent (s : Story) : Prop :=
  ∀ e ∈ s.events, ∀ ca ∈ e.declaredAges, ∀ b, s.birthOf ca.1 = some b →
    ca.2 = fullYears b e.date

theorem agesCoherent_sound (s : Story) (h : agesCoherentB s = true) : AgesCoherent s := by
  intro e he ca hca b hb
  simp only [agesCoherentB, List.all_eq_true] at h
  have := h e he ca hca
  simp_all [ageOk]

/-! ## Invariant 3 – no bilocation

A character is not present in two different places on the same day. -/

def sharesParticipant (a b : Event) : Bool :=
  a.participants.any fun c => b.participants.contains c

```

```

def noBilocationB (s : Story) : Bool :=
  datesConsistent s && s.events.all fun a => s.events.all fun b =>
    !(a.day == b.day && a.place != b.place && sharesParticipant a b)

def NoBilocation (s : Story) : Prop :=
  ∀ a ∈ s.events, ∀ b ∈ s.events, ∀ c, c ∈ a.participants → c ∈ b.participants →
    a.date.dayNumber = b.date.dayNumber → a.place = b.place

theorem noBilocation_sound (s : Story) (h : noBilocationB s = true) : NoBilocation s := by
  intro a ha b hb c hca hcb hday
  simp only [noBilocationB, Bool.and_eq_true, List.all_eq_true] at h
  have hab := h.2 a ha b hb
  rw [← day_eq_of_consistent h.1 ha, ← day_eq_of_consistent h.1 hb] at hday
  have hsh : sharesParticipant a b = true := by
    simp only [sharesParticipant, List.any_eq_true, List.contains_iff_mem]
    exact {c, hca, hcb}
  simp_all

/-! ## Invariant 4 – nobody acts after leaving

After an event of kind `death` or `departure`, its first participant takes part in no
event told later. The outer loop runs over exit events only, so the check costs
(exits × events), not events2. -/

def noAfterExitB (s : Story) : Bool :=
  (s.events.filter (·.kind.isExit)).all fun x => s.events.all fun y =>
    !(decide (x.seq < y.seq) &&
      (x.participants.head?.map (fun c => y.participants.contains c)).getD false)

def NoAfterExit (s : Story) : Prop :=
  ∀ x ∈ s.events, ∀ y ∈ s.events, ∀ c, x.kind.isExit = true → x.participants.head? = some c →
    x.seq < y.seq → c ∉ y.participants

theorem noAfterExit_sound (s : Story) (h : noAfterExitB s = true) : NoAfterExit s := by
  intro x hx y hy c hk hc hlt hmem
  simp only [noAfterExitB, List.all_eq_true] at h
  have := h x (List.mem_filter.mpr {hx, hk}) y hy
  simp_all

/-! ## The whole story -/

def validStory (s : Story) : Bool :=
  temporalOrderB s && agesCoherentB s && noBilocationB s && noAfterExitB s

def ValidStory (s : Story) : Prop :=
  TemporalOrder s ∧ AgesCoherent s ∧ NoBilocation s ∧ NoAfterExit s

theorem validStory_sound (s : Story) (h : validStory s = true) : ValidStory s := by
  simp only [validStory, Bool.and_eq_true] at h
  exact {temporalOrder_sound s h.1.1.1, agesCoherent_sound s h.1.1.2,
    noBilocation_sound s h.1.2, noAfterExit_sound s h.2}

end Chronology

```

Cronologías de ejemplo

Fuente: `formal/lean/examples/`

ok.json (pasa)

```
{
  "novel_id": "sample-ok",
  "characters": [
    {"id": "c1", "name": "Lucía", "birth_date": "1990-05-12"},
    {"id": "c2", "name": "Tomás \"el Viejo\"", "birth_date": "1948-11-30"},
    {"id": "c3", "name": "Marta", "birth_date": null}
  ],
  "places": [
    {"id": "p1", "name": "Valencia"},
    {"id": "p2", "name": "Madrid"}
  ],
  "events": [
    {"id": "e1", "seq": 1, "story_date": "2020-05-11", "place_id": "p1", "participants": ["c1", "c2"],
    "kind": "normal", "declared_ages": {"c1": 29, "c2": 71}, "chapter": 1, "description": "The day before
her thirtieth birthday."},
    {"id": "e2", "seq": 2, "story_date": "2020-05-12", "place_id": "p1", "participants": ["c1"],
    "kind": "normal", "declared_ages": {"c1": 30}, "chapter": 1, "description": "Her birthday."},
    {"id": "e3", "seq": 3, "story_date": "2020-05-12", "place_id": "p2", "participants": ["c3"],
    "kind": "normal", "chapter": 2, "description": "Same day, another city, another person."},
    {"id": "e4", "seq": 4, "story_date": "2020-12-01", "place_id": "p1", "participants": ["c2", "c1"],
    "kind": "death", "declared_ages": {"c2": 72}, "chapter": 3, "description": "Tomás dies."},
    {"id": "e5", "seq": 5, "story_date": "2021-01-15", "place_id": "p2", "participants": ["c1", "c3"],
    "kind": "departure", "chapter": 4, "description": "Lucía leaves."}
  ]
}
```

incoherent.json (falla agesCoherent y noBilocation)

```
{
  "novel_id": "sample-incoherent",
  "characters": [
    {"id": "c1", "name": "Lucía", "birth_date": "1990-05-12"},
    {"id": "c2", "name": "Tomás", "birth_date": "1948-11-30"}
  ],
  "places": [
    {"id": "p1", "name": "Valencia"},
    {"id": "p2", "name": "Madrid"}
  ],
  "events": [
    {"id": "e1", "seq": 1, "story_date": "2020-05-11", "place_id": "p1", "participants": ["c1"],
    "kind": "normal", "declared_ages": {"c1": 30}, "chapter": 1, "description": "Declared 30 one day
before the birthday: aged wrong."},
    {"id": "e2", "seq": 2, "story_date": "2020-06-01", "place_id": "p1", "participants": ["c1", "c2"],
    "kind": "normal", "chapter": 2, "description": "Lunch in Valencia."},
    {"id": "e3", "seq": 3, "story_date": "2020-06-01", "place_id": "p2", "participants": ["c2"],
    "kind": "normal", "chapter": 2, "description": "Same day, Tomás also in Madrid: bilocated."}
  ]
}
```